

Microsoft Fabric의 실시간 대시보드 시작하기

Microsoft Fabric의 실시간 대시보드를 사용하면 Kusto Query Language(KQL)를 사용하여 스트리밍 데이터를 시각화하고 탐색할 수 있습니다. 이 실습에서는 실시간 데이터 소스를 기반으로 실시간 대시보드를 만들고 사용하는 방법을 탐색합니다.

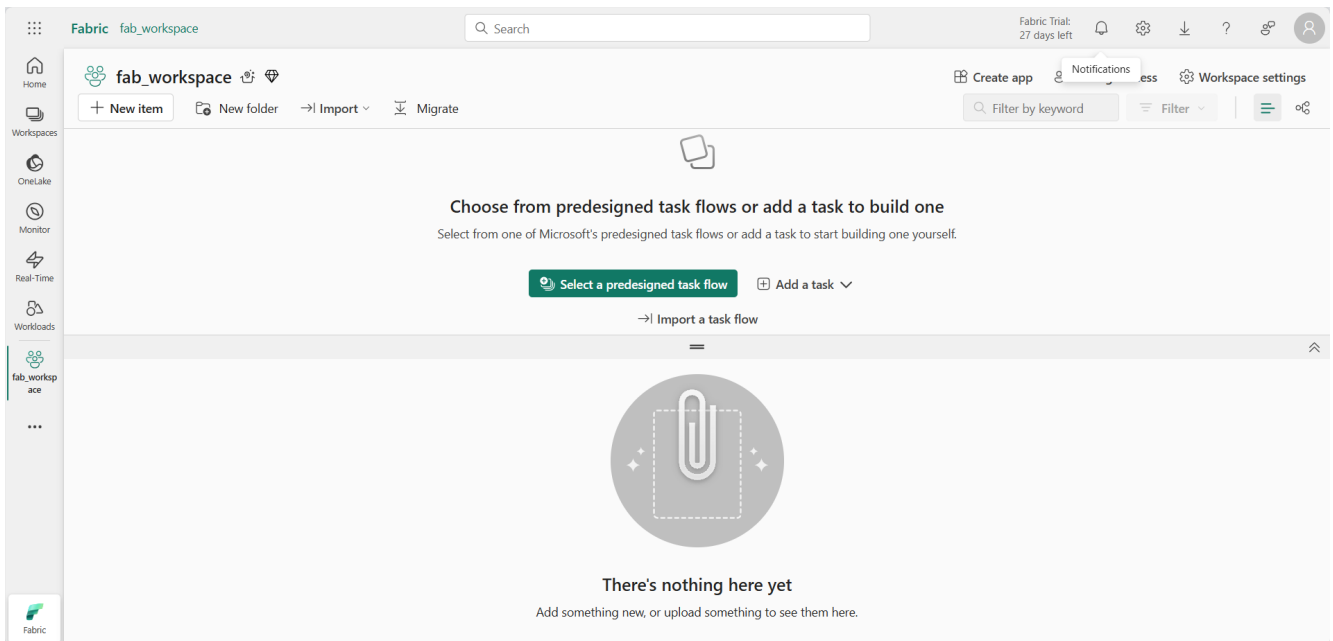
이 실습을 완료하는 데는 약 **40분**이 소요됩니다.

Note: 이 실습을 완료하려면 [Microsoft Fabric 테넌트](#)가 필요합니다.

Workspace 생성

Fabric에서 데이터를 다루기 전에, Fabric 용량이 활성화된 Workspace(작업 영역)를 생성해야 합니다.

1. 브라우저에서 [Microsoft Fabric 홈페이지](https://app.fabric.microsoft.com/home?experience=fabric) (<https://app.fabric.microsoft.com/home?experience=fabric>)로 이동하여 Fabric 자격 증명으로 로그인합니다.
2. 왼쪽 메뉴 바에서 **Workspaces** (아이콘 모양 )를 선택합니다.
3. 원하는 이름으로 새 Workspace를 만들고, Fabric 용량을 포함하는 라이선스 모드(*Trial*, *Premium*, 또는 *Fabric*)를 선택합니다.
4. 새 Workspace가 열리면 비어 있어야 합니다.



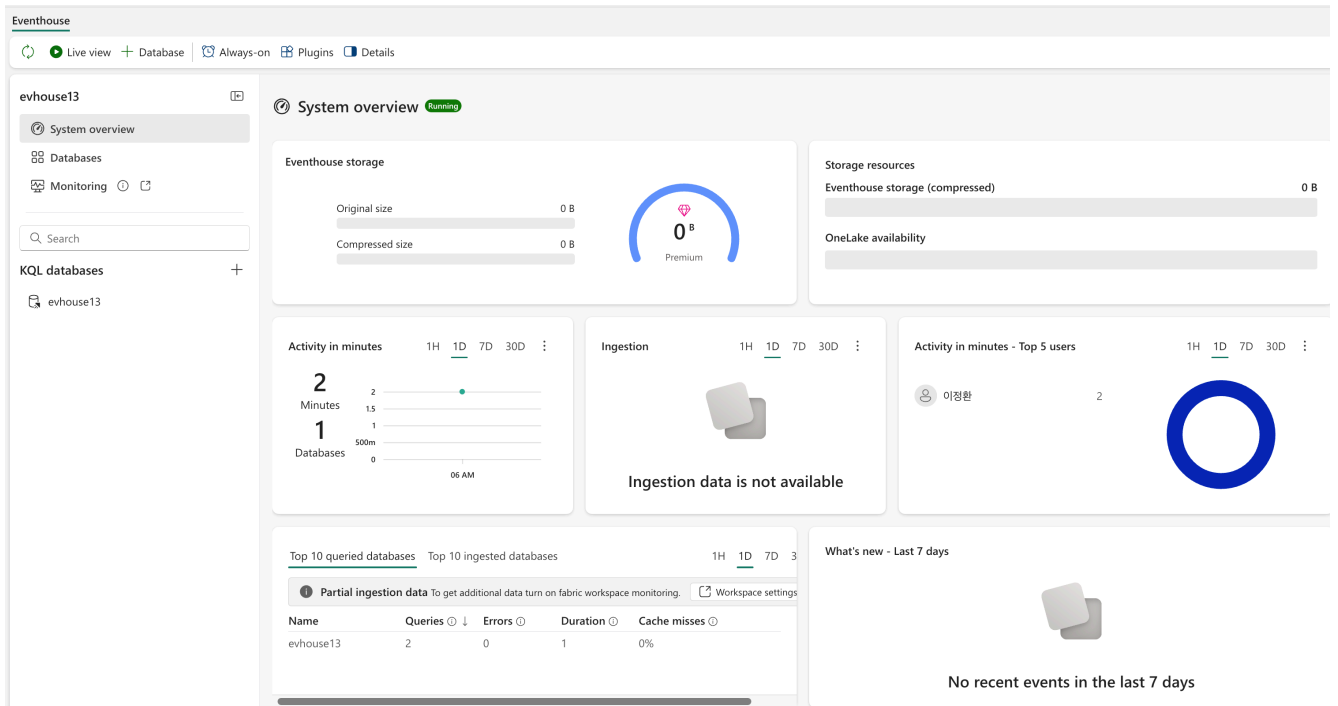
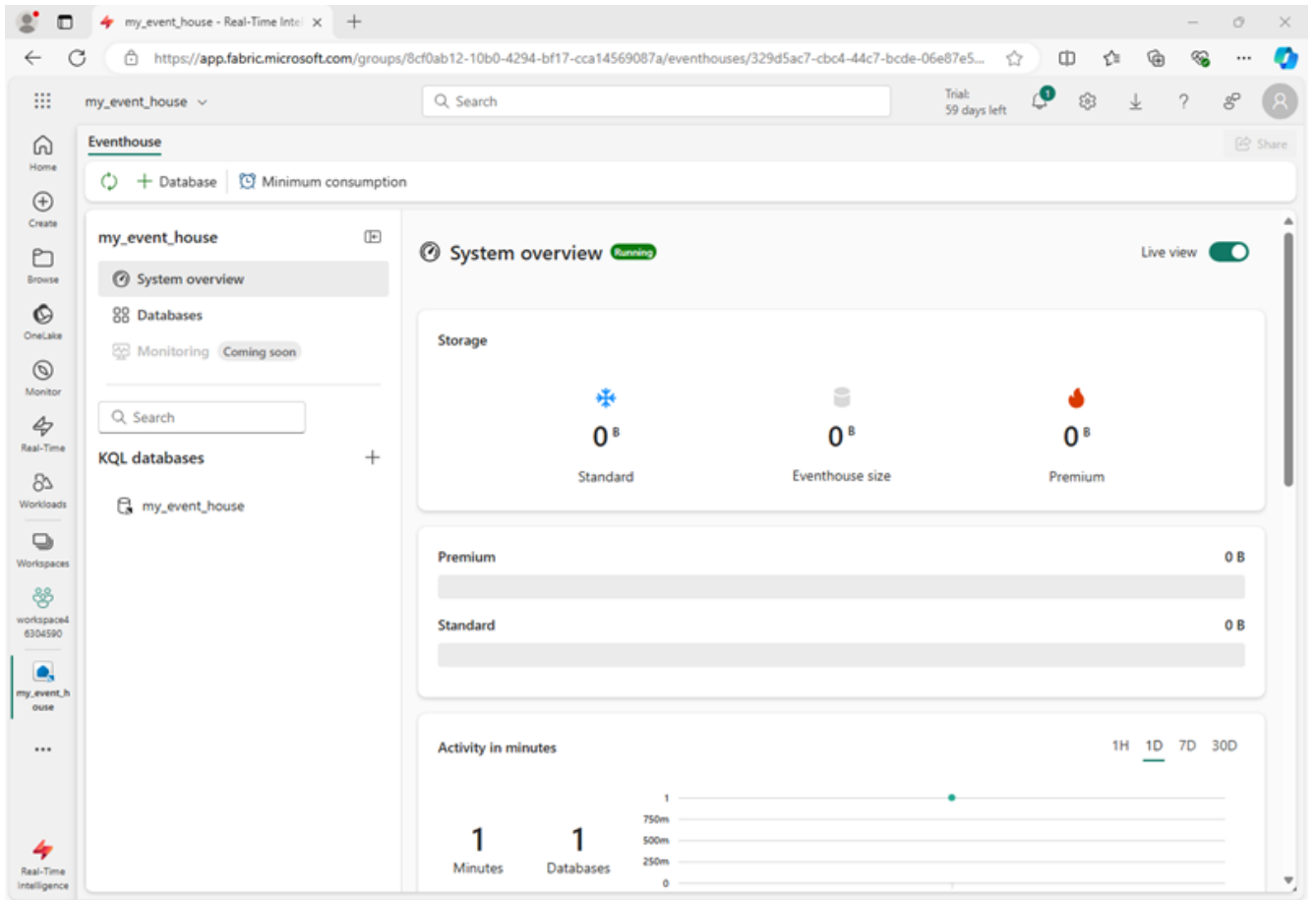
Eventhouse 생성

이제 Workspace가 있으므로 실시간 인텔리전스 솔루션에 필요한 Fabric 항목 생성을 시작할 수 있습니다. 먼저 **eventhouse**를 생성하여 시작하겠습니다.

1. 왼쪽 메뉴 바에서 **Create**를 선택합니다. *New* 페이지의 *Real-Time Intelligence* 섹션에서 **Eventhouse**를 선택합니다. 원하는 고유한 이름을 지정합니다.

Note: **Create** 옵션이 사이드바에 고정되어 있지 않으면 먼저 줄임표(...) 옵션을 선택해야 합니다.

2. 새로운 빈 **eventhouse**가 보일 때까지 표시되는 모든 팁이나 프롬프트를 닫습니다.



3. 왼쪽 창에서 eventhouse가 eventhouse와 동일한 이름의 KQL database를 포함하고 있는지 확인합니다.

4. KQL database를 선택하여 봅니다.

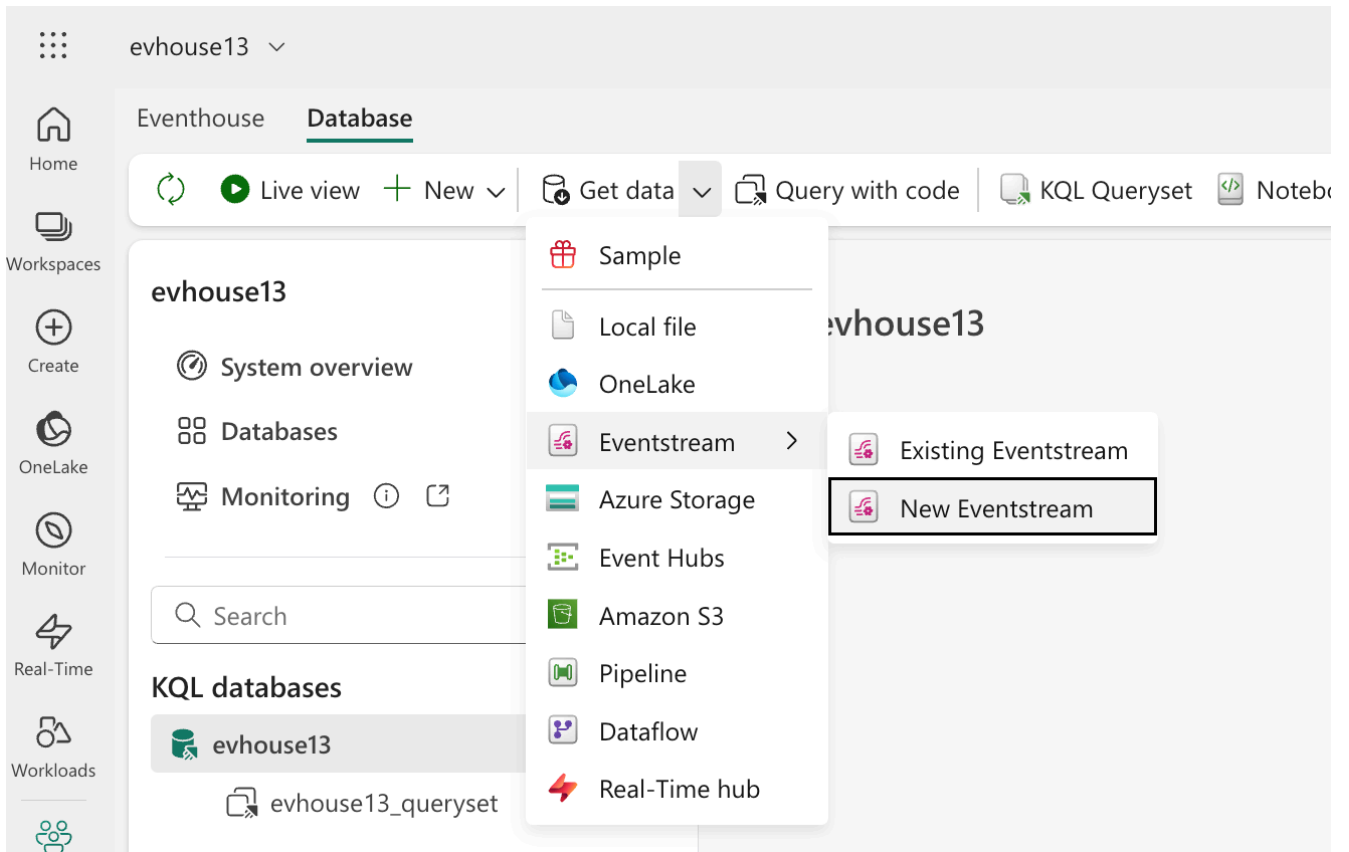
Eventstream 생성

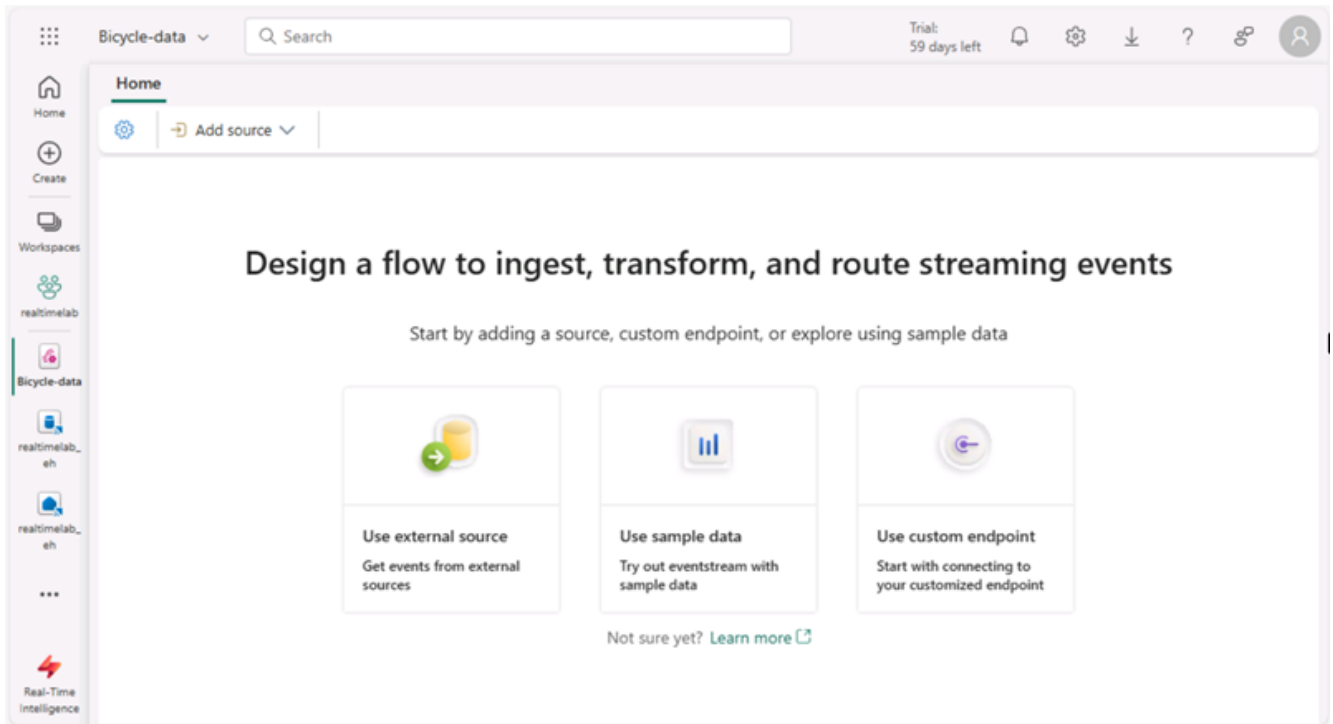
개념 설명: Eventstream

- **Eventstream**은 실시간 데이터를 Fabric으로 가져오는(Ingestion) 파이프라인 역할을 합니다. 다양한 소스(예: Azure Event Hubs, IoT Hub 또는 이 실습에서처럼 제공되는 샘플 데이터)로부터 지속적으로 들어오는 데이터 스트림을 캡처하여, **KQL Database**나 Lakehouse와 같은 Fabric 내의 여러 대상으로 라우팅할 수 있습니다. 즉, **Eventstream**은 실시간 데이터의 '입구' 역할을 하며, 데이터가 발생하는 즉시 분석할 수 있도록 준비해 줍니다.

현재 데이터베이스에는 테이블이 없습니다. **eventstream**을 사용하여 실시간 소스에서 테이블로 데이터를 로드하겠습니다.

1. KQL 데이터베이스의 메인 페이지에서 **Get data**를 선택합니다.
2. 데이터 소스로 **Eventstream > New eventstream**을 선택합니다. **eventstream**의 이름을 **Bicycle-data**로 지정합니다.





Workspace에 새 `eventstream`이 곧 생성됩니다. 생성이 완료되면 자동으로 `eventstream`의 데이터 소스를 선택하는 화면으로 리디렉션됩니다.

3. **Use sample data**를 선택합니다.

4. 소스 이름으로 `Bicycles`를 지정하고, **Bicycles** 샘플 데이터를 선택합니다.

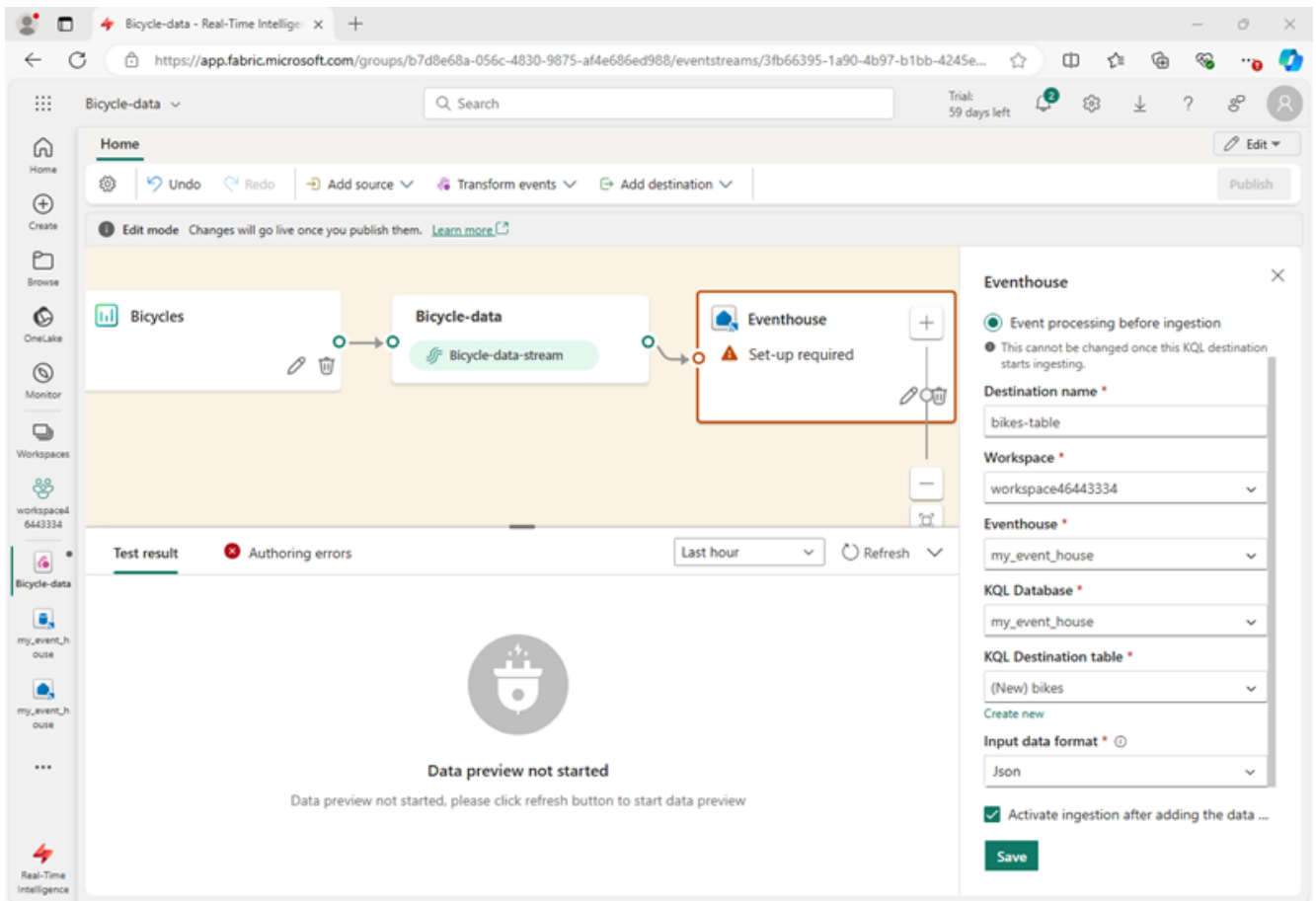
스트림이 매핑되고 **eventstream canvas**에 자동으로 표시됩니다.

BikepointID	Street	Neighbourhood	Latitude	Longitude	No_Bikes
BikePoints_517	Ford Road	Old Ford	51.5325127	-0.033085	12
BikePoints_471	Hewison Street	Old Ford	51.5332832	-0.028155	12
BikePoints_459	Gunmakers Lane	Old Ford	51.53518	-0.03338	2
BikePoints_590	Greenberry Street	St. John's Wood	51.53256	-0.16862	11
BikePoints_394	Aberdeen Place	St. John's Wood	51.524826	-0.176268	1
BikePoints_363	Lord's	St. John's Wood	51.52912	-0.171185	20
BikePoints_312	Grove End Road	St. John's Wood	51.5308876	-0.17677	1
BikePoints_286	St. John's Wood Road	St. John's Wood	51.5272942	-0.174653	25
BikePoints_110	Wellington Road	St. John's Wood	51.5330429	-0.172528	8

5. **Add destination** 드롭다운 목록에서 **Eventhouse**를 선택합니다.

6. **Eventhouse** 창에서 다음 설정 옵션을 구성합니다.

- **Data ingestion mode:** Event processing before ingestion
- **Destination name:** bikes-table
- **Workspace:** 이 실습 시작 시 생성한 Workspace를 선택합니다
- **Eventhouse:** 생성한 eventhouse를 선택합니다
- **KQL database:** 생성한 KQL database를 선택합니다
- **Destination table:** (create new 선택) bikes 라는 이름의 새 테이블 생성
- **Input data format:** JSON



7. **Eventhouse** 창에서 **Save**를 선택합니다.

8. **Bicycles-data** 노드의 출력을 **bikes-table** 노드에 연결한 다음 **Publish**를 선택합니다.

9. 데이터 대상이 활성화될 때까지 1분 정도 기다립니다. 그런 다음 디자인 캔버스에서 **bikes-table** 노드를 선택하고 아래의 **Data preview** 창을 보며 수집된 최신 데이터를 확인합니다.

The screenshot shows the Microsoft Fabric Real-Time Intelligence interface. The top navigation bar includes a search bar and a 'Live' button. The left sidebar contains a 'Data' section with 'Sources' (Bicycles) and 'Destinations' (bikes-table). The main area displays a data stream diagram with a 'Bicycle-data' stream and a 'bikes-table' destination. Below the diagram, the 'Data preview' tab is active, showing a table of bike data.

BikepointID	Street	Neighbourhood	Latitude	Longitude
BikePoints_680	Westbridge Road	Battersea	51.477684	-0.170329
BikePoints_670	Ashley Crescent	Battersea	51.467186	-0.152248
BikePoints_627	Holden Street	Battersea	51.46679	-0.15823
BikePoints_651	Thorndike Close	West Chelsea	51.4809	-0.183853
BikePoints_573	Limerston Street	West Chelsea	51.4855881	-0.18119
BikePoints_529	Manresa Road	Chelsea	51.4868927	-0.170983
BikePoints_395	Cadogan Gardens	Chelsea	51.4924622	-0.159919
BikePoints_250	Royal Avenue 1	Chelsea	51.489933	-0.162727
BikePoints_345	Flood Street	Chelsea	51.48802	-0.166878

10. 몇 분 기다린 후 **Refresh** 버튼을 사용하여 **Data preview** 창을 새로고침합니다. 스트림은 계속 실행되므로 새 데이터가 테이블에 추가되었을 수 있습니다.

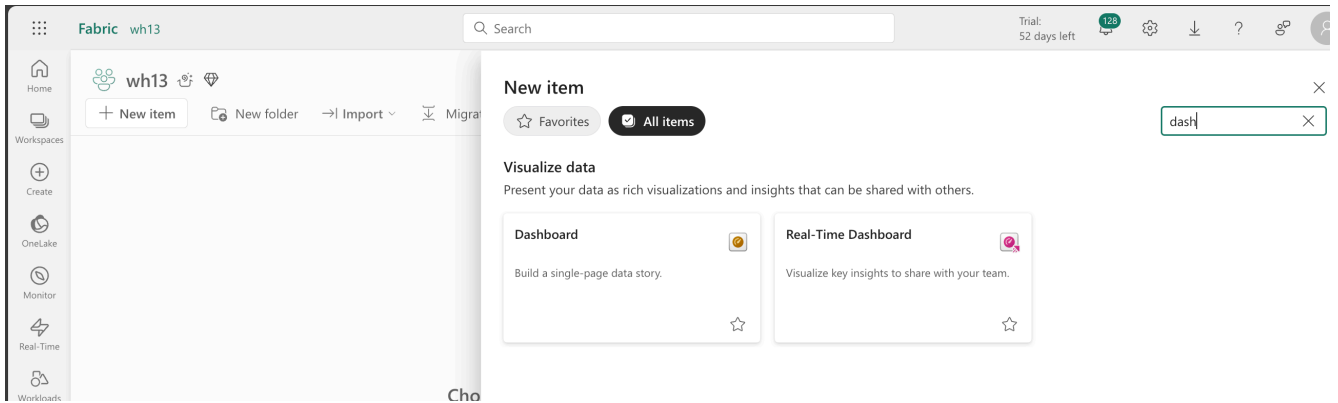
실시간 대시보드 생성

개념 설명: Real-Time Dashboard

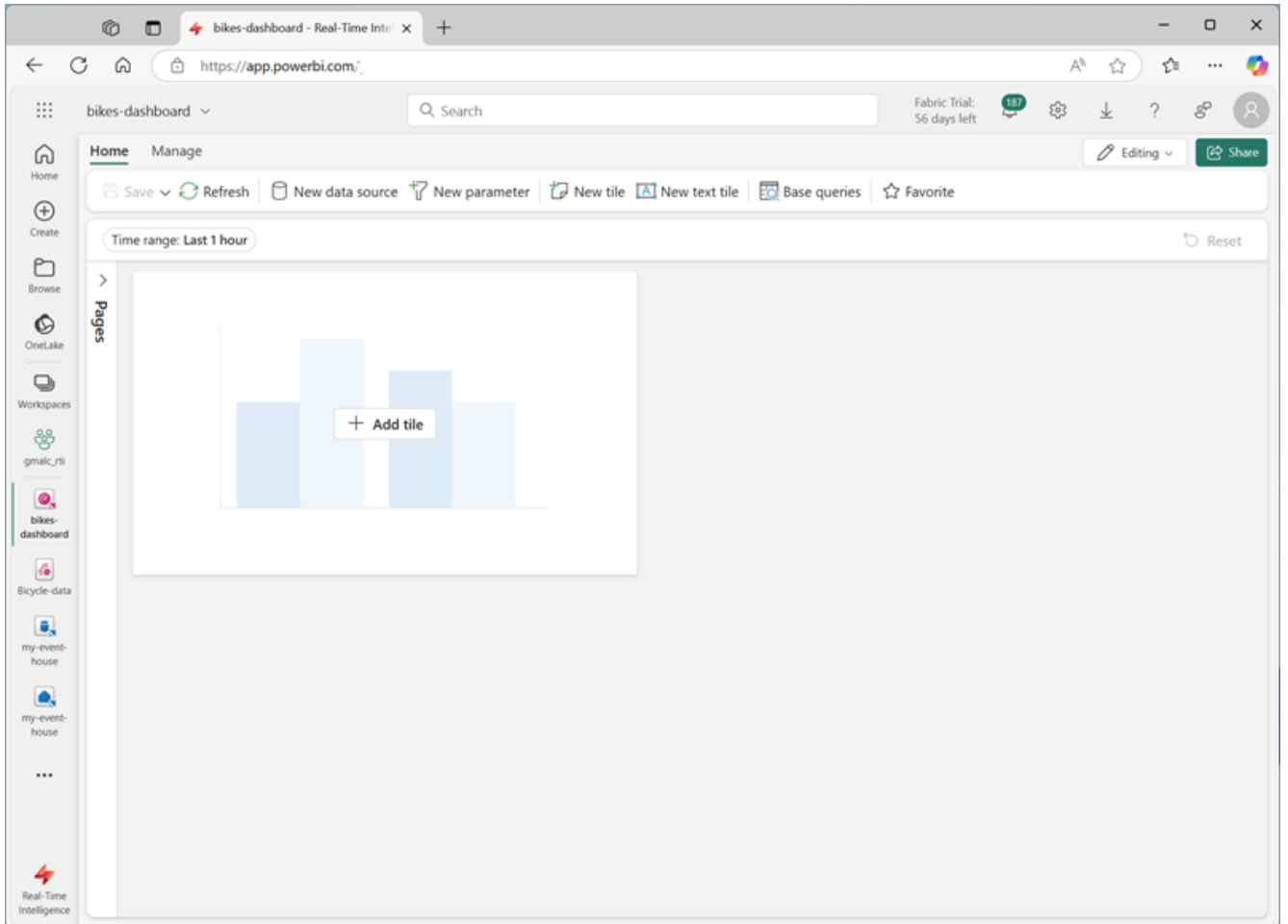
- 실시간 대시보드는 **KQL Database**에 저장된 데이터를 시각화하는 데 특화된 도구입니다. Power BI 대시보드와 달리, 실시간 대시보드는 **KQL 쿼리**를 직접 사용하여 데이터를 조회하고 시각화하며, 데이터가 거의 실시간으로 업데이트되는 시나리오(예: 모니터링, 운영 분석)에 최적화되어 있습니다. 각 시각적 요소(타일)는 개별 KQL 쿼리에 의해 구동되며, 대시보드 전체적으로 자동 새로고침 기능을 설정하여 최신 데이터를 지속적으로 표시할 수 있습니다.

이제 **eventhouse**의 테이블로 실시간 데이터 스트림이 로드되었으므로, 실시간 대시보드로 시각화할 수 있습니다.

- 왼쪽 메뉴 바에서 **Home** 허브를 선택합니다. 그런 다음 홈페이지에서 **bikes-dashboard**라는 이름의 새 **Real-Time Dashboard**를 생성합니다.



새로운 빈 대시보드가 생성됩니다.



2. 톨바에서 **New data source**를 선택하고 새로운 **One lake data hub** 데이터 소스를 추가합니다. 그런 다음 **eventhouse**를 선택하고 다음 설정으로 새 데이터 소스를 만듭니다.
 - **Display name:** `Bike Rental Data`
 - **Database:** *eventhouse*의 기본 데이터베이스.
 - **Passthrough identity:** 선택됨
3. **Data sources** 창을 닫은 다음, 대시보드 디자인 캔버스에서 **Add tile**을 선택합니다.
4. 쿼리 편집기에서 **Bike Rental Data** 소스가 선택되었는지 확인하고 다음 KQL 코드를 입력합니다.

```
bikes
| where ingestion_time() between (ago(30min) .. now())
| summarize latest_observation = arg_max(ingestion_time(), *) by Neighbourhood
| project Neighbourhood, latest_observation, No_Bikes, No_Empty_Docks
| order by Neighbourhood asc
```

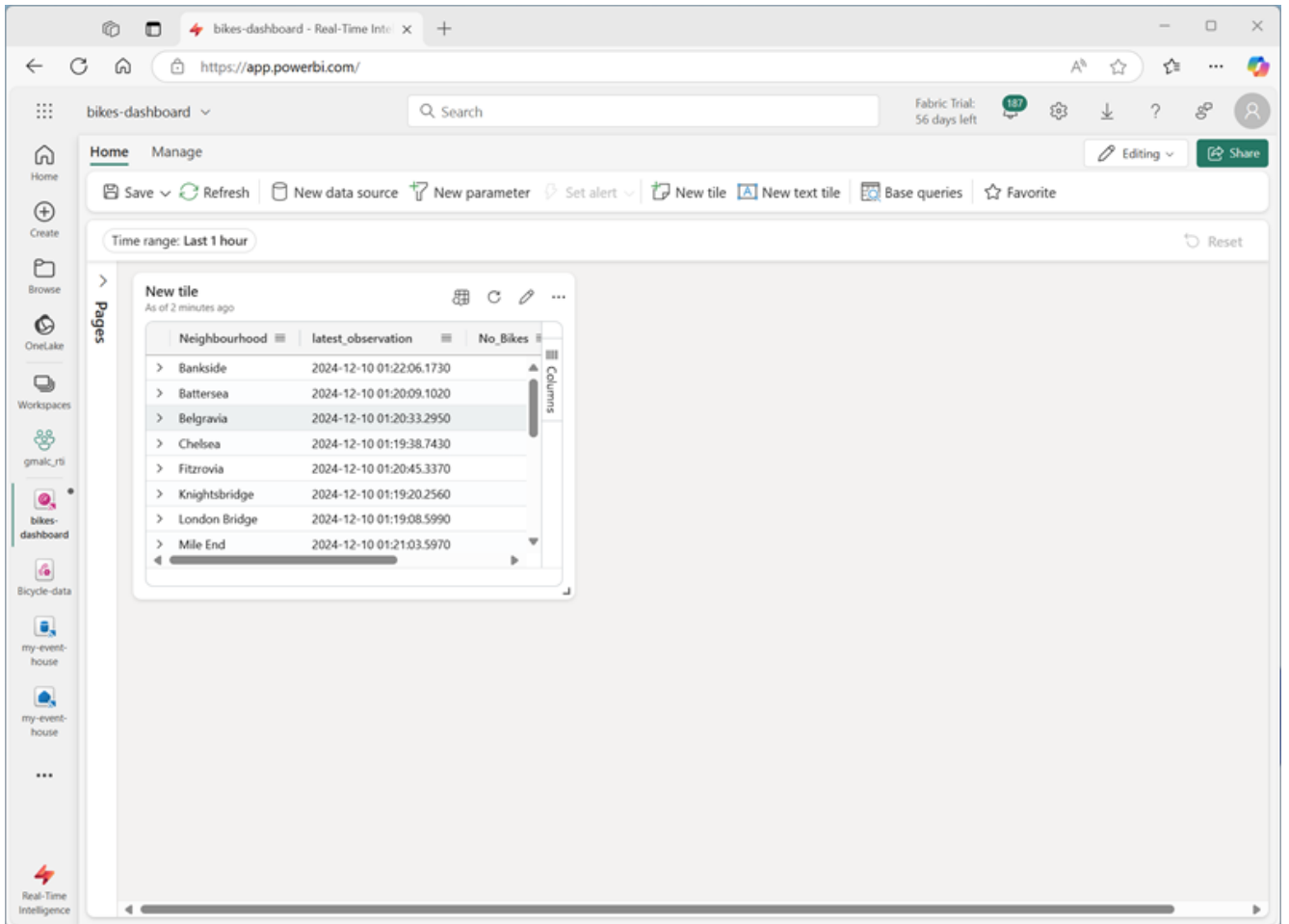
코드 설명:

- `bikes`: 쿼리할 테이블 이름입니다.
- `| where ingestion_time() between (ago(30min) .. now())`: `ingestion_time()` 함수를 사용하여 데이터가 수집된 시간을 기준으로 필터링합니다. `ago(30min)`는 '30분 전'을 의미하므로, 이 구문은 지난 30분 동안 수집된 데이터만 선택합니다.
- `| summarize latest_observation = arg_max(ingestion_time(), *) by Neighbourhood`: 데이터를 `Neighbourhood` 별로 그룹화하고, 각 그룹 내에서 가장 최근에 수집된(`ingestion_time()`이 가장 큰) 행의 모든 열(`*`)을 찾습니다. `arg_max`는 "최대값의 인수"를 찾는 함수로, 이 경우 "각 동네의 가장 마지막 데이터"를 찾는 데 사용됩니다.
- `| project Neighbourhood, latest_observation, No_Bikes, No_Empty_Docks`: 결과에 표시할 열을 선택합니다.
- `| order by Neighbourhood asc`: 결과를 동네 이름의 오름차순으로 정렬합니다.

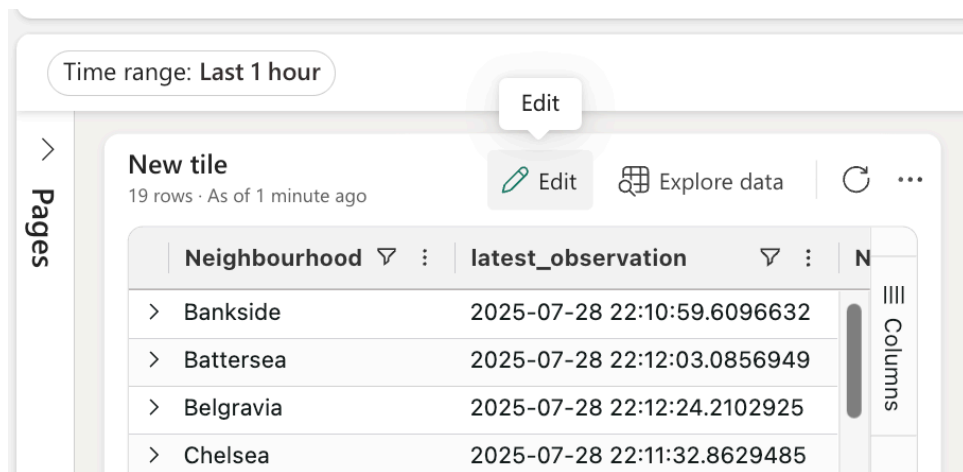
5. 쿼리를 실행하면 지난 30분 동안 각 동네에서 관찰된 자전거 수와 빈 자전거 거치대 수가 표시됩니다.

6. 변경 사항을 적용(Apply Changes)하여 대시보드의 타일에 데이터가 테이블 형태로 표시되도록 합니다.





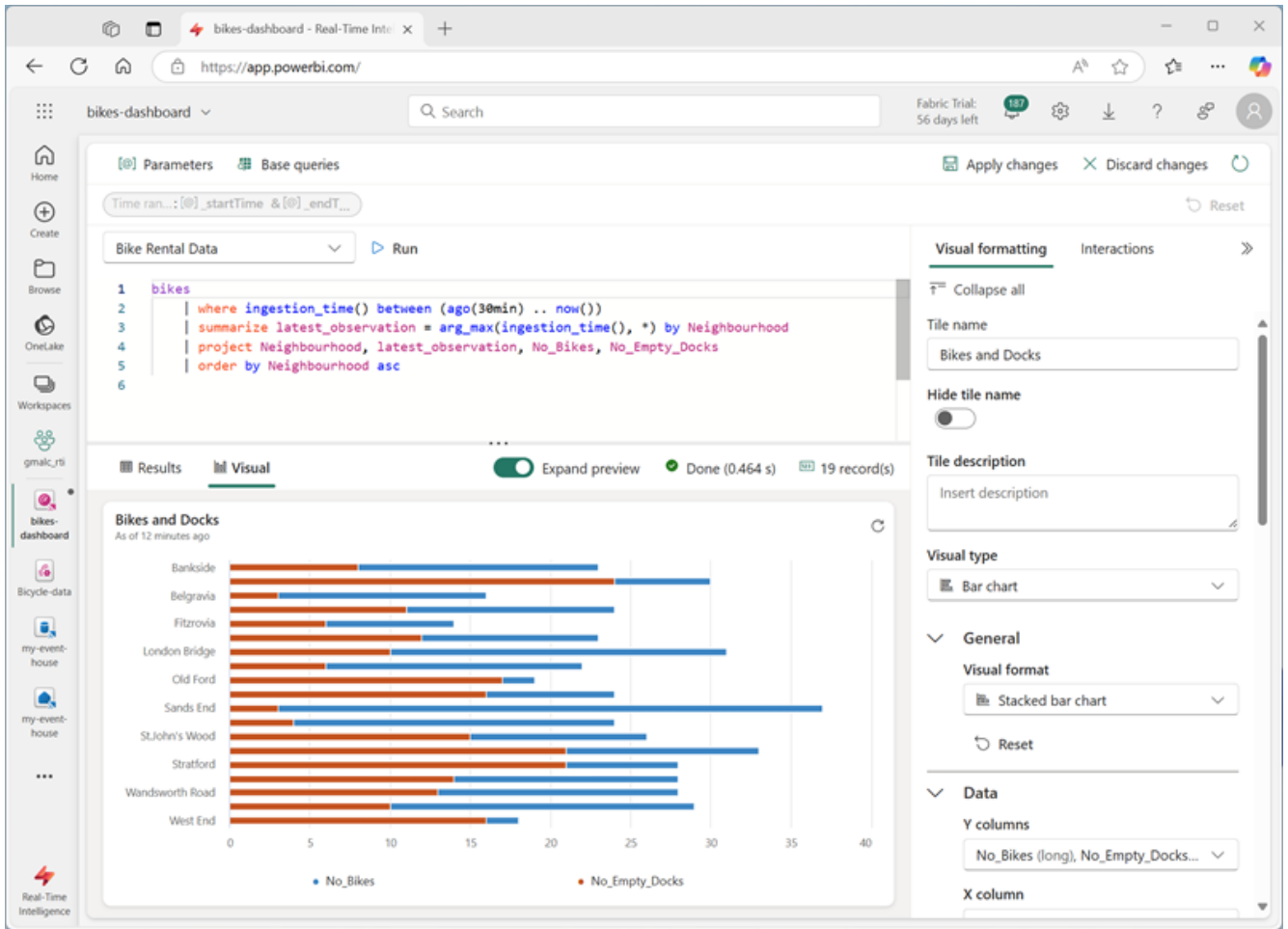
7. 타일에서 **Edit** 아이콘(연필 모양)을 선택합니다.



그런 다음 **Visual Formatting** 창에서 다음 속성을 설정합니다.

- **Tile name:** Bikes and Docks
- **Visual type:** Bar chart
- **Visual format:** Stacked bar chart
- **Y columns:** No_Bikes, No-Empty_Docks
- **X column:** Neighbourhood
- **Series columns:** infer
- **Legend location:** Bottom

편집된 타일은 다음과 같아야 합니다.



8. 변경 사항을 적용한 다음, 타일의 크기를 조정하여 대시보드 왼쪽 전체 높이를 차지하도록 합니다.
9. 톨바에서 **New tile**을 선택합니다.
10. 쿼리 편집기에서 **Bike Rental Data** 소스가 선택되었는지 확인하고 다음 KQL 코드를 입력합니다.

```
bikes
| where ingestion_time() between (ago(30min) .. now())
| summarize latest_observation = arg_max(ingestion_time(), *) by Neighbourhood
| project Neighbourhood, latest_observation, Latitude, Longitude, No_Bikes
| order by Neighbourhood asc
```

코드 설명:

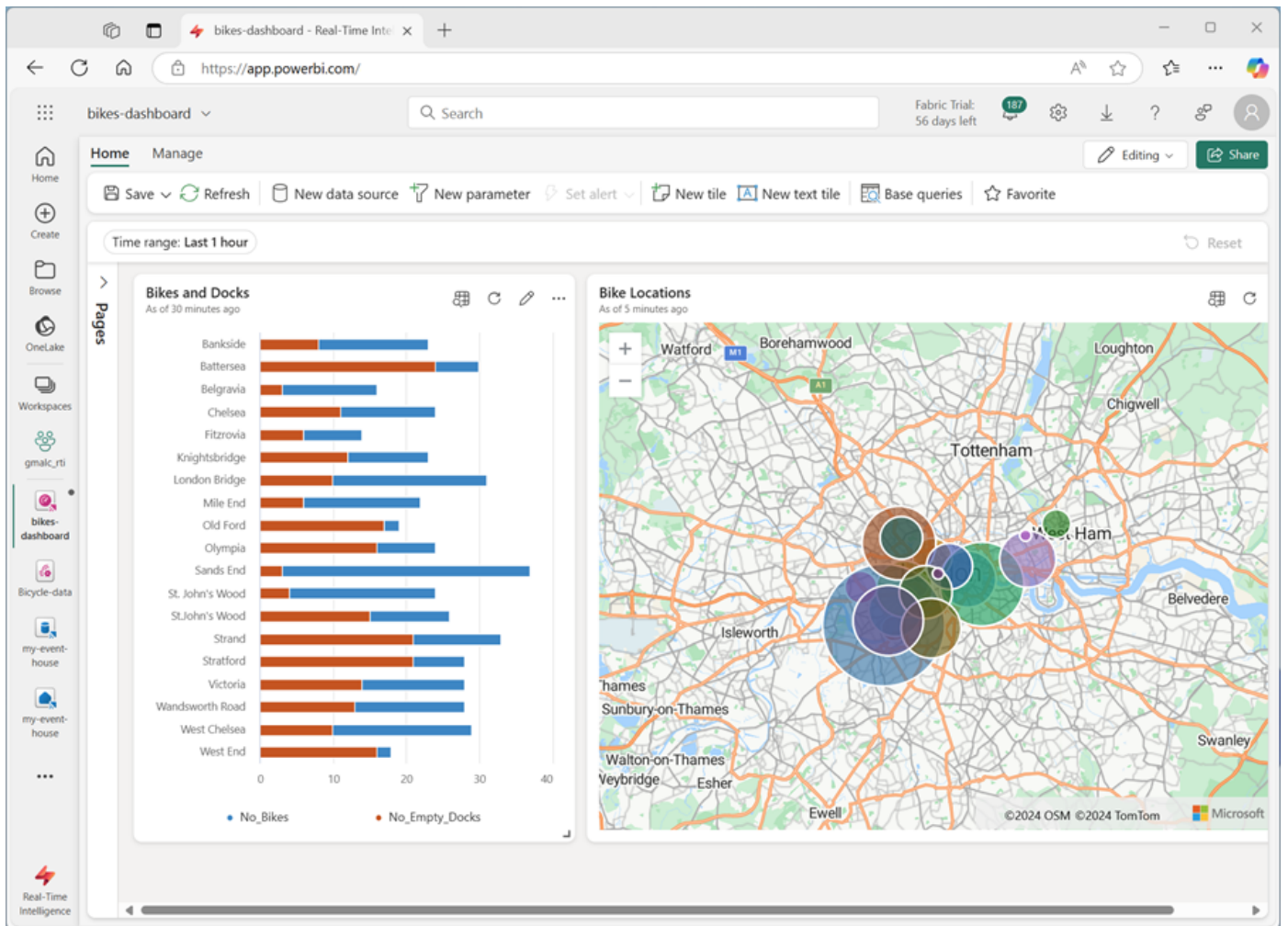
이 쿼리는 이전 쿼리와 거의 동일하지만, 지도 시각화를 위해 `project` 절에서 위도(`Latitude`)와 경도(`Longitude`) 열을 선택한다는 점이 다릅니다.

11. 쿼리를 실행하면 지난 30분 동안 각 동네에서 관찰된 위치와 자전거 수가 표시됩니다.
12. 변경 사항을 적용하여 대시보드의 타일에 데이터가 테이블 형태로 표시되도록 합니다.
13. 타일에서 **Edit** 아이콘(연필 모양)을 선택합니다. 그런 다음 **Visual Formatting** 창에서 다음 속성을 설정합니다.

- **Tile name:** `Bike Locations`
- **Visual type:** `Map`
- **Define location by:** `Latitude and longitude`

- **Latitude column:** Latitude
- **Longitude column:** Longitude
- **Label column:** Neighbourhood
- **Size:** Show
- **Size column:** No_Bikes

14. 변경 사항을 적용한 다음, 지도 타일의 크기를 조정하여 대시보드에서 사용 가능한 공간의 오른쪽을 채웁니다.



기본 쿼리 생성

개념 설명: Base Query

Base query (기본 쿼리)는 대시보드 내의 여러 타일(시각적 요소)에서 공통적으로 사용할 수 있는 데이터셋을 미리 정의하는 기능입니다. 두 개 이상의 타일이 유사한 데이터(예: 동일한 필터링 및 집계)를 필요로 할 때, 중복된 쿼리를 각 타일에 작성하는 대신 하나의 **base query**를 만들고 각 타일에서는 이 **base query**의 결과를 참조하여 필요한 추가 작업(예: **project**로 열 선택)만 수행합니다.

장점:

- **유지보수성 향상:** 공통 로직이 변경되어야 할 때, **base query** 하나만 수정하면 이를 사용하는 모든 타일에 변경사항이 적용됩니다.

- **성능 최적화:** 대시보드는 공통 쿼리를 한 번만 실행하여 결과를 캐시하고 여러 타일에서 재사용하므로, 전체적인 쿼리 부하가 줄어듭니다.

대시보드에는 유사한 쿼리를 기반으로 하는 두 개의 시각적 요소가 포함되어 있습니다. 중복을 피하고 대시보드의 유지보수성을 높이기 위해 공통 데이터를 단일 `base query`로 통합할 수 있습니다.

1. 대시보드 툴바에서 **Base queries**를 선택한 다음 **+Add**를 선택합니다.
2. 기본 쿼리 편집기에서 **Variable name**을 `base_bike_data`로 설정하고 **Bike Rental Data** 소스가 선택되었는지 확인합니다. 그런 다음 다음 쿼리를 입력합니다.

```
bikes
| where ingestion_time() between (ago(30min) .. now())
| summarize latest_observation = arg_max(ingestion_time(), *) by Neighbourhood
```

코드 설명:

이 쿼리는 두 타일에서 공통적으로 사용되던 "지난 30분 동안 각 동네의 최신 데이터 가져오기" 로직을 그대로 담고 있습니다.

3. 쿼리를 실행하고 대시보드의 두 시각적 요소에 필요한 모든 열(및 기타 열)을 반환하는지 확인합니다.

Neighbourhood	latest_observation	No_Bikes	No_Empty_Docks	Latitude	Longitude
> Bankside	2024-12-10 02:22:37.2980	15	8	51.50541	-0.098341
> Battersea	2024-12-10 02:23:31.8790	8	28	51.46517	-0.180389
> Belgravia	2024-12-10 02:21:03.4260	13	18	51.49456	-0.153933
> Chelsea	2024-12-10 02:23:10.7060	14	10	51.4851227	-0.174971
> Fitzrovia	2024-12-10 02:21:15.5980	24	6	51.5209923	-0.139016
> Knightsbridge	2024-12-10 02:22:51.9120	11	12	51.50143	-0.178656
> London Bridge	2024-12-10 02:22:40.2240	23	8	51.50215	-0.083632
> Mile End	2024-12-10 02:21:33.7300	15	7	51.51854	-0.034903
> Old Ford	2024-12-10 02:22:25.1230	3	15	51.5360374	-0.026262
> Olympia	2024-12-10 02:20:45.2840	8	16	51.5000877	-0.211316
> Sands End	2024-12-10 02:21:42.0950	34	3	51.4751434	-0.187278
> St. John's Wood	2024-12-10 02:22:16.2000	21	3	51.52912	-0.171185
> StJohn's Wood	2024-12-10 02:22:19.2430	11	15	51.53256	-0.16862

4. **Done**을 선택한 다음 **Base queries** 창을 닫습니다.
5. **Bikes and Docks** 막대 차트 시각적 요소를 편집하고 쿼리를 다음 코드로 변경합니다.

```
base_bike_data
| project Neighbourhood, latest_observation, No_Bikes, No_Empty_Docks
| order by Neighbourhood asc
```

코드 설명:

이제 쿼리는 테이블 이름(`bikes`) 대신 `base_bike_data` 변수로부터 시작합니다. 복잡한 `where`와 `summarize` 로직이 사라지고, 단순히 필요한 열을 선택(`project`)하고 정렬하는 작업만 남아 쿼리가 훨씬 간결해졌습니다.

6. 변경 사항을 적용하고 막대 차트가 여전히 모든 동네에 대한 데이터를 표시하는지 확인합니다.

7. **Bike Locations** 지도 시각적 요소를 편집하고 쿼리를 다음 코드로 변경합니다.

```
base_bike_data
| project Neighbourhood, latest_observation, No_Bikes, Latitude, Longitude
| order by Neighbourhood asc
```

코드 설명:

이 쿼리 역시 `base_bike_data`를 사용하여 필요한 열만 `project` 합니다.

8. 변경 사항을 적용하고 지도가 여전히 모든 동네에 대한 데이터를 표시하는지 확인합니다.

매개변수 추가

개념 설명: Parameters

- `Parameter` (매개변수)는 대시보드를 동적이고 상호작용적으로 만드는 기능입니다. 사용자는 드롭다운 목록이나 텍스트 상자와 같은 컨트롤을 통해 값을 선택할 수 있으며, 이 선택된 값은 대시보드 쿼리(주로 `base query`)에 전달되어 결과를 필터링하는 데 사용됩니다. 이를 통해 사용자는 코드를 직접 수정하지 않고도 보고 싶은 데이터의 범위를 직접 제어할 수 있습니다.

현재 대시보드는 모든 동네에 대한 최신 자전거, 거치대 및 위치 데이터를 보여줍니다. 이제 특정 동네를 선택할 수 있도록 매개변수를 추가해 보겠습니다.

- 대시보드 툴바의 **Manage** 탭에서 **Parameters**를 선택합니다.
- 자동으로 생성된 기존 매개변수(예: *Time range* 매개변수)가 있다면 모두 **Delete**합니다.
- + Add**를 선택합니다.
- 다음 설정으로 매개변수를 추가합니다.
 - **Label:** `Neighbourhood`
 - **Parameter type:** `Multiple selection` (다중 선택 가능)
 - **Description:** `Choose neighbourhoods`
 - **Variable name:** `selected_neighbourhoods` (쿼리에서 사용할 변수 이름)
 - **Data type:** `string`

- **Show on pages:** `Select all` (모든 페이지에 이 매개변수 표시)
- **Source:** `Query` (쿼리 결과를 통해 선택 목록 생성)
- **Data source:** `Bike Rental Data`
- **Edit query:**

```
bikes
| distinct Neighbourhood
| order by Neighbourhood asc
```

이 쿼리는 `bikes` 테이블에서 중복되지 않는(`distinct`) 동네 이름 목록을 가져와 오름차순으로 정렬합니다. 이 결과가 사용자에게 보여질 드롭다운 목록의 항목이 됩니다.

- **Value column:** `Neighbourhood`
- **Label column:** `Match value selection`
- **Add "Select all" value:** 선택됨 ("모두 선택" 옵션 추가)
- **"Select all" sends empty string:** 선택됨 ("모두 선택" 시 빈 문자열 전달)
- **Auto-reset to default value:** 선택됨
- **Default value:** `Select all`

5. **Done**을 선택하여 매개변수를 생성합니다.

이제 매개변수를 추가했으므로, 선택한 동네에 따라 데이터를 필터링하도록 기본 쿼리를 수정해야 합니다.

6. 툴바에서 **Base queries**를 선택합니다. 그런 다음 **base_bike_data** 쿼리를 선택하고 다음 코드와 같이 선택된 매개변수 값에 따라 필터링하도록 **where** 절에 **and** 조건을 추가하여 편집합니다.

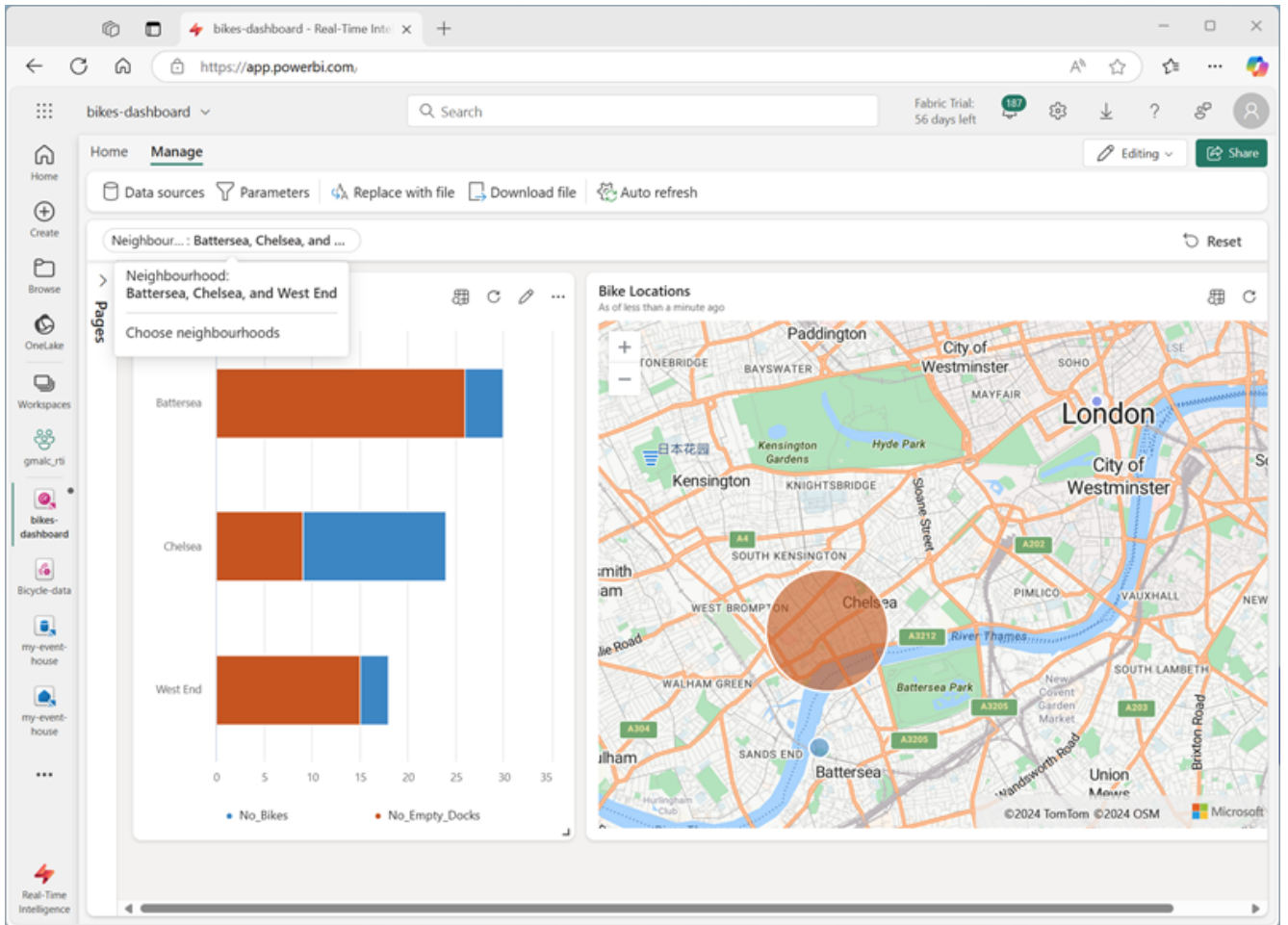
```
bikes
| where ingestion_time() between (ago(30min) .. now())
    and (isempty(['selected_neighbourhoods']) or Neighbourhood in
(['selected_neighbourhoods']))
| summarize latest_observation = arg_max(ingestion_time(), *) by Neighbourhood
```

코드 설명:

- `and (isempty(['selected_neighbourhoods']) or Neighbourhood in (['selected_neighbourhoods']))`: 이 구문은 매개변수를 효과적으로 사용하는 핵심 패턴입니다.
 - `isempty(['selected_neighbourhoods'])`: 만약 사용자가 "모두 선택"을 했거나 아무것도 선택하지 않아 매개변수 값이 비어있으면 `true`가 됩니다.
 - `Neighbourhood in (['selected_neighbourhoods'])`: 만약 사용자가 하나 이상의 동네를 선택했다면, `Neighbourhood` 열의 값이 선택된 목록에 포함된 경우에만 `true`가 됩니다.
 - `or` 연산자로 연결되어 있으므로, 아무것도 선택하지 않으면 모든 동네가 표시되고, 무언가를 선택하면 해당 동네만 필터링됩니다.

7. **Done**을 선택하여 기본 쿼리를 저장합니다.

8. 대시보드에서 **Neighbourhood** 매개변수를 사용하여 선택한 동네에 따라 데이터를 필터링합니다.

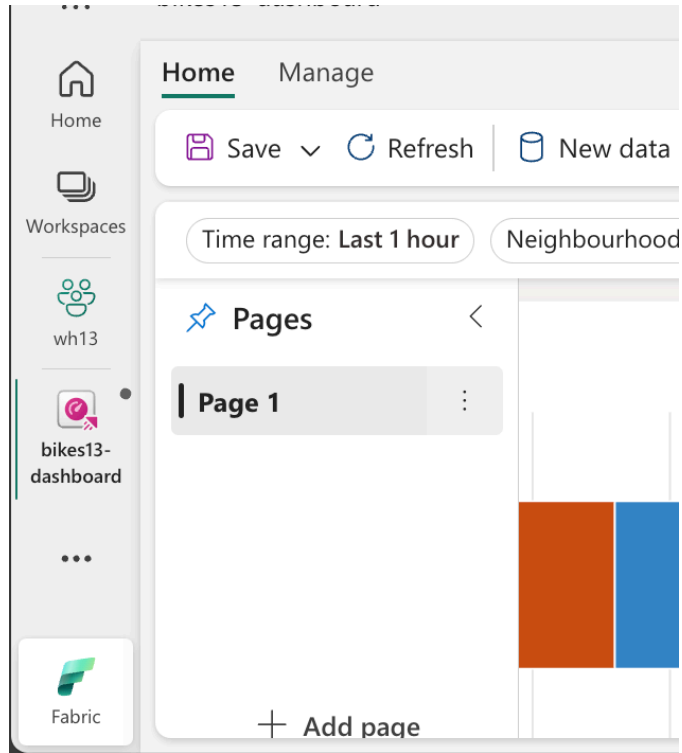


9. **Reset**을 선택하여 선택된 매개변수 필터를 제거합니다.

페이지 추가

현재 대시보드는 단일 페이지로 구성되어 있습니다. 더 많은 데이터를 제공하기 위해 페이지를 더 추가할 수 있습니다.

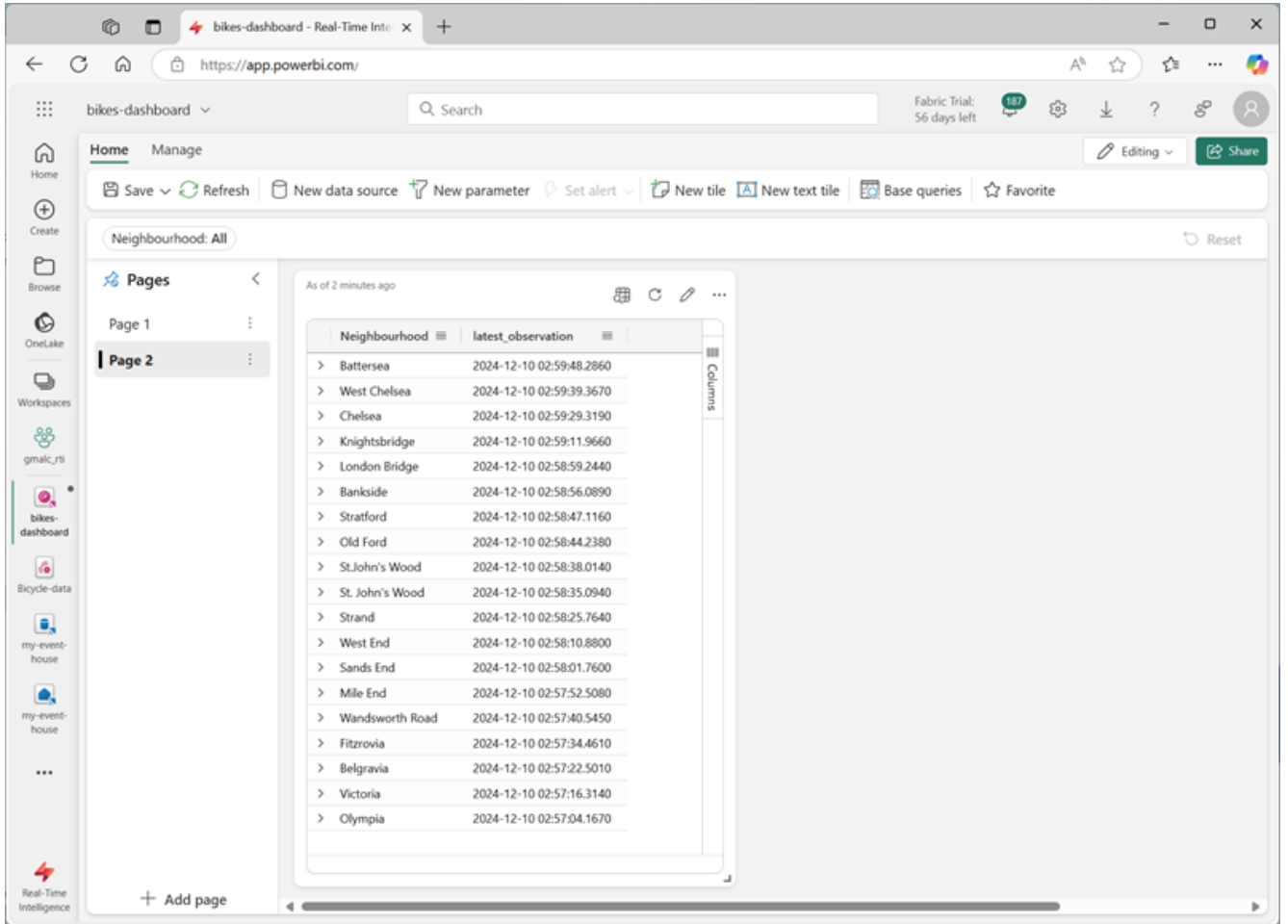
1. 대시보드 왼쪽에서 **Pages** 창을 확장하고 **+ Add page**를 선택합니다.



2. 새 페이지의 이름을 **Page 2**로 지정한 다음 선택합니다.
3. 새 페이지에서 **+ Add tile**을 선택합니다.
4. 새 타일의 쿼리 편집기에 다음 쿼리를 입력합니다.

```
base_bike_data  
| project Neighbourhood, latest_observation  
| order by latest_observation desc
```

5. 변경 사항을 적용합니다. 그런 다음 타일 크기를 조정하여 대시보드 높이를 채웁니다.



자동 새로고침 구성

사용자가 수동으로 대시보드를 새로고침할 수 있지만, 설정된 간격으로 데이터를 자동으로 새로고침하도록 하는 것이 유용할 수 있습니다.

1. 대시보드 톨바의 **Manage** 탭에서 **Auto refresh**를 선택합니다.
2. **Auto refresh** 창에서 다음 설정을 구성합니다.
 - **Enabled:** 선택됨
 - **Minimum time interval:** Allow all refresh intervals
 - **Default refresh rate:** 30 minutes
3. 자동 새로고침 설정을 적용합니다.

대시보드 저장 및 공유

이제 유용한 대시보드가 있으므로 저장하고 다른 사용자와 공유할 수 있습니다.

1. 대시보드 톨바에서 **Save**를 선택합니다.
2. 대시보드가 저장되면 **Share**를 선택합니다.
3. **Share** 대화 상자에서 **Copy link**를 선택하고 대시보드 링크를 클립보드에 복사합니다.
4. 새 브라우저 탭을 열고 복사한 링크를 붙여넣어 공유된 대시보드로 이동합니다. 메시지가 표시되면 Fabric 자격 증명으로 다시 로그인합니다.

5. 대시보드를 탐색하여 도시 전역의 자전거 및 빈 자전거 거치대에 대한 최신 정보를 확인합니다.

리소스 정리

대시보드 탐색을 마쳤다면, 이 실습을 위해 생성한 Workspace를 삭제할 수 있습니다.

1. 왼쪽 바에서 Workspace의 **아이콘**을 선택합니다.
2. 툴바에서 **Workspace settings**를 선택합니다.
3. **General** 섹션에서 **Remove this workspace**를 선택합니다.